PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL)

Complete Report — 10 Selected Case Studies

Institution	Dr. RVR NRI Institute of Technology
Programme	B. Tech – CSE (AIML)
Student Name	P. THANUSHA
Roll Number	26001107
Branch / Section	CSM / 4
Course	Python Programming
GitHub Repository	Python-Activity-Based-Learning

This report contains the required documentation for each selected case study. Replace every marked screenshot area with your own readable output screenshot after running the corresponding Python file.

Report Guide

Mandatory sections included for every case: Problem Statement, Objectives, Assumptions, Algorithm, Flowchart, Pseudocode, Python Source Code, Test Cases, Output Screenshot(s), and Conclusion.

Selected case studies

Case	Case Study	Python file
12	Attendance Recovery Computation	case12_attendance.py
13	ATM Transaction Sequence Simulation	case13_atm.py
23	Student Timetable Conflict Detection	case23_timetable.py
28	Repeated Transaction Detection	case28_transactions.py
51	Bus Passenger Occupancy Simulation	case51_bus_occupancy.py
57	Smart Canteen Order and Stock Management	case57_canteen.py
66	Student Academic Risk Classification	case66_academic_risk.py
87	Examination Question Difficulty and Performance Analysis	case87_exam_analysis.py
91	Advanced String-Based Registration Record Analyzer	case91_registration.py
93	Smart Public Transport Fare Reconciliation	case93_fare_reconciliation.py

Screenshot instructions

Run each program using your own Python environment and insert your own output screenshot(s) in the blank area provided in that case. If two test cases fit in one screenshot, that is acceptable as long as the input and output are readable. Simple filenames such as case12.png, case13.png, case23.png, case28.png, case51.png, case57.png, case66.png, case87.png, case91.png, and case93.png can be used.

Case 12: Attendance Recovery Computation

Python file: **case12_attendance.py**

1. Problem Statement

Calculate current attendance percentage and find the minimum number of additional classes a student must attend to reach a target attendance percentage.

2. Objectives

• Calculate the current attendance percentage.

• Use a while loop to find the minimum additional classes required.

• Validate class counts and target percentage.

3. Assumptions

• Total classes must be greater than zero.

• Attended classes must be between zero and total classes.

• The student attends every future class.

• The target percentage must be greater than 0 and at most 100.

4. Algorithm

1. Start.

2. Read total classes, attended classes, and target percentage.

3. Validate the inputs.

4. Calculate current attendance percentage.

5. If the target is already met, report that result.

6. Otherwise, increase attended and total classes one at a time until the target is met.

7. Display the result and stop.

5. Flowchart

START → Read inputs → Validate? → Calculate attendance → Target met? → If no, repeat adding one attended class until target reached → Display result → STOP

6. Pseudocode

```
1. Read total, attended, and target.
2. If class details or target are invalid, display an error.
3. Otherwise calculate attendance = attended / total × 100.
4. If attendance ≥ target, report target achieved.
5. Else set required = 0 and increment required until (attended + required) / (total + required) × 100 ≥ target.
6. Print attendance and required classes.
```

7. Python Source Code

```
total = int(input("Enter total classes: ")) attended =
int(input("Enter classes attended: ")) target =
float(input("Enter target attendance percentage: ")) if total
<= 0 or attended < 0 or attended > total:
    print("Invalid class details") elif
target <= 0 or target > 100:
    print("Invalid target percentage") else:
    attendance = attended / total * 100     print("Current
attendance:", round(attendance, 2), "%")     if attendance
>= target:
        print("Target attendance already achieved")
else:
        required = 0         while ((attended + required) / (total +
required)) * 100 < target:
            required += 1
        print("Additional classes required:", required)
```

8. Test Cases

• Test 1: 40, 26, 75 -> Current attendance 65.0%; additional classes required 16

• Test 2: 20, 16, 75 -> Current attendance 80.0%; target already achieved

• Test 3: 40, 45, 75 -> Invalid class details

• Test 4: 40, 26, 110 -> Invalid target percentage

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program calculates current attendance and determines the minimum additional classes needed to reach the target, while validating the inputs.

Case 13: ATM Transaction Sequence Simulation

Python file: **case13_atm.py**

1. Problem Statement

Simulate a sequence of deposits and withdrawals, update an account balance, and count successful and rejected transactions.

2. Objectives

• Process multiple transactions using a loop.

• Update the balance for deposits and withdrawals.

• Reject invalid amounts, invalid transaction types, and withdrawals beyond the balance.

3. Assumptions

• The opening balance is supplied by the user.

• Transaction amounts must be positive.

• Only deposit and withdraw actions are supported.

• No overdraft is allowed.

4. Algorithm

1. Start.

2. Read opening balance and number of transactions.

3. For each transaction, read action and amount.

4. Validate amount and action.

5. Update balance for a valid deposit or withdrawal; otherwise reject it.

6. Count successful and rejected transactions.

7. Display final balance and counts.

5. Flowchart

START → Read balance and transaction count → Repeat for each transaction → Validate action/amount → Update balance or reject → Display summary → STOP

6. Pseudocode

```
1. Read opening balance and number of transactions.
2. Repeat for each transaction: read action and amount.
3. If amount ≤ 0, reject it.
4. Else if action is deposit, add amount to balance.
5. Else if action is withdraw, subtract if sufficient balance; otherwise reject.
6. Reject any other action.
7. Print final balance and transaction counts.
```

7. Python Source Code

```
balance = float(input("Enter opening balance: "))
n = int(input("Enter number of transactions: "))
successful = rejected = 0 for i in range(n):
    print("Transaction", i + 1)
    action = input("Enter deposit or withdraw: ").lower()
amount = float(input("Enter amount: "))     if amount <=
0:         print("Invalid amount")         rejected += 1
elif action == "deposit":
        balance += amount
print("Deposit accepted")
successful += 1     elif action ==
"withdraw":         if amount <= balance:
balance -= amount
print("Withdrawal accepted")
successful += 1         else:
            print("Insufficient balance")
rejected += 1     else:
        print("Invalid transaction type")
rejected += 1 print("Final balance:",
round(balance, 2)) print("Successful
```

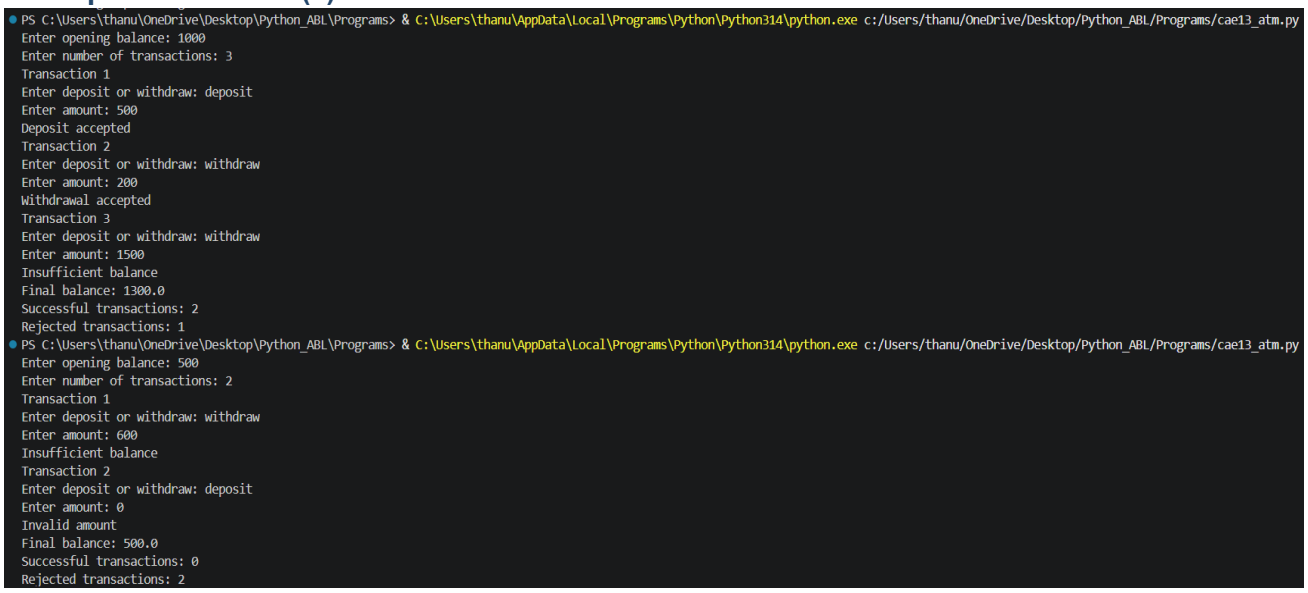
```
transactions:", successful) print("Rejected
transactions:", rejected)
```

8. Test Cases

• Test 1: Balance 1000; deposit 500, withdraw 200, withdraw 1500 -> final 1300.0; successful 2; rejected 1

• Test 2: Balance 500; withdraw 600, deposit 0 -> final 500.0; successful 0; rejected 2

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program simulates deposits and withdrawals, maintains the account balance, and reports successful and rejected transactions.

Case 23: Student Timetable Conflict Detection

Python file: **case23_timetable.py**

1. Problem Statement

Compare class time intervals and report pairs of classes that overlap on the same day.

2. Objectives

• Store class details in a list of tuples.

• Use nested loops to compare class pairs.

• Detect overlapping intervals on the same day.

3. Assumptions

• Days are entered as text and compared without case sensitivity.

• Start time must be less than end time.

• Time is entered in whole hours.

• Two classes conflict only when their day matches and their time intervals overlap.

4. Algorithm

1. Start.

2. Read number of classes.

3. For each class, read subject, day, start hour, and end hour.

4. Validate time; skip invalid records.

5. Compare each pair of classes.

6. If day matches and intervals overlap, report a conflict.

7. If no conflicts were found, display that result.

8. Stop.

5. Flowchart

START → Read classes → Validate each time interval → Compare each pair → Same day and overlapping times? → Report conflict / no conflict → STOP

6. Pseudocode

```
1. Read the number of classes.
2. For each class, read subject, day, start, and end.
3. Skip invalid time intervals.
4. For each pair (i, j) where j > i, unpack the class tuples.
5. If days match and start1 < end2 and start2 < end1, report a conflict.
6. If no conflicts were found, display the no-conflict message.
```

7. Python Source Code

```
n = int(input("Enter number of classes: "))
classes = [] for i in range(n):
    print("Class", i + 1)     subject = input("Enter
subject name: ")     day = input("Enter day:
").upper()     start = int(input("Enter start hour
(0-23): "))     end = int(input("Enter end hour (0-
23): "))     if start < 0 or end > 23 or start >=
end:         print("Invalid time; record skipped")
continue     classes.append((subject, day, start,
end)) conflict_found = False for i in
range(len(classes)):     for j in range(i + 1,
len(classes)):
        s1, d1, a1, b1 = classes[i]
s2, d2, a2, b2 = classes[j]         if d1 ==
d2 and a1 < b2 and a2 < b1:
            print("Conflict:", s1, "and", s2, "on", d1)
conflict_found = True if not conflict_found:
    print("No timetable conflicts detected")
```

8. Test Cases

• Test 1: Math MON 9–11; Python MON 10–12 -> conflict detected

• Test 2: Math MON 9–10; Python MON 10–11 -> no conflict

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program compares class intervals on the same day and identifies timetable clashes using nested loops and tuple records.

Case 28: Repeated Transaction Detection

Python file: **case28_transactions.py**

1. Problem Statement

Check transaction records for repeated transaction IDs and identify transactions whose amounts exceed a chosen threshold.

2. Objectives

• Detect duplicate transaction IDs.

• Identify high-value transactions.

• Display totals for repeated and high-value transactions.

3. Assumptions

• Transaction IDs are compared after trimming spaces and converting to uppercase.

• A transaction is high-value only when its amount is strictly greater than the threshold.

• Amounts are entered as numbers.

4. Algorithm

1. Start.

2. Read number of transactions and threshold.

3. Create an empty set and set both counters to zero.

4. For each transaction, read ID and amount.

5. Check whether ID has appeared before; update duplicate count.

6. Check whether amount exceeds threshold.

7. Display both counts and stop.

5. Flowchart

START → Read count and threshold → Read each ID and amount → ID already seen? → Count duplicate if yes; otherwise save ID → Amount above threshold? → Count if yes → Print summary → STOP ## 6. Pseudocode

```
1. Read number of transactions and high-value threshold.
2. Create an empty set for seen IDs and initialise counters.
3. For each transaction, normalise the ID and read amount.
4. If ID is already in the set, report and count a repeat; otherwise add it.
5. If amount > threshold, report and count it.
6. Print repeated and high-value totals.
```

7. Python Source Code

```
n = int(input("Enter number of transactions: ")) threshold
= float(input("Enter high-value threshold: ")) seen =
set()
duplicates = high_value = 0 for
i in range(n):
    print("Transaction", i + 1)     transaction_id = input("Enter
transaction ID: ").strip().upper()     amount = float(input("Enter
transaction amount: "))     if transaction_id in seen:
        print("Repeated transaction ID:", transaction_id)
duplicates += 1     else:
        seen.add(transaction_id)     if amount >
threshold:         print("High-value
transaction:", amount)         high_value += 1
print("Repeated transactions:", duplicates)
print("High-value transactions:", high_value)
```

8. Test Cases

• Test 1: Threshold 10000; T101 500, T102 200, T102 12000 -> repeated 1; high-value 1

• Test 2: Threshold 10000; T201 500, T202 10000 -> repeated 0; high-value 0

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program detects repeated transaction IDs and high-value transactions, then displays summary counts.

Case 51: Bus Passenger Occupancy Simulation

Python file: **case51_bus_occupancy.py**

1. Problem Statement

Track passenger boarding and alighting at bus stops, report the current occupancy, and warn if capacity is exceeded.

2. Objectives

• Update passenger count at every stop.• Validate boarding and alighting values.

• Detect when the bus exceeds its capacity.

3. Assumptions

• Initial passenger count is zero.

• Capacity and stop count must be positive.

• Passenger counts cannot be negative.

• Alighting passengers cannot exceed the current passenger count.

4. Algorithm

1. Start.
2. Read bus capacity and number of stops.
3. Validate both values.
4. For each stop, read boarding and alighting counts.
5. Reject invalid passenger details.
6. Update current passengers.
7. Report occupancy and capacity warnings.
8. Display final passenger count and stop.

5. Flowchart

START → Read capacity/stops → Valid? → For each stop read boarding/alighting → Validate → Update occupancy → Capacity exceeded? → Display final count → STOP

6. Pseudocode

```
1. Read capacity and number of stops; initialise passengers to zero.
2. If capacity or stops is not positive, display an error.
3. For each stop, read boarding and alighting values.
4. If values are invalid or alighting exceeds current passengers, report invalid details and continue.
5. Update passengers = passengers - alighting + boarding.
6. Display current count and warn if it exceeds capacity.
7. Print final passenger count.
```

7. Python Source Code

```
capacity = int(input("Enter bus capacity: "))
stops = int(input("Enter number of stops: "))
passengers = 0 if capacity <= 0 or stops <= 0:
    print("Invalid capacity or number of stops") else:     for i in
range(stops):         print("Stop", i + 1)         boarding =
int(input("Passengers boarding: "))         alighting =
int(input("Passengers alighting: "))          if boarding < 0 or
alighting < 0 or alighting > passengers:
            print("Invalid passenger details")
continue         passengers = passengers - alighting +
boarding         print("Current passengers:",
passengers)          if passengers > capacity:
            print("Capacity exceeded")
print("Final passenger count:", passengers)
```

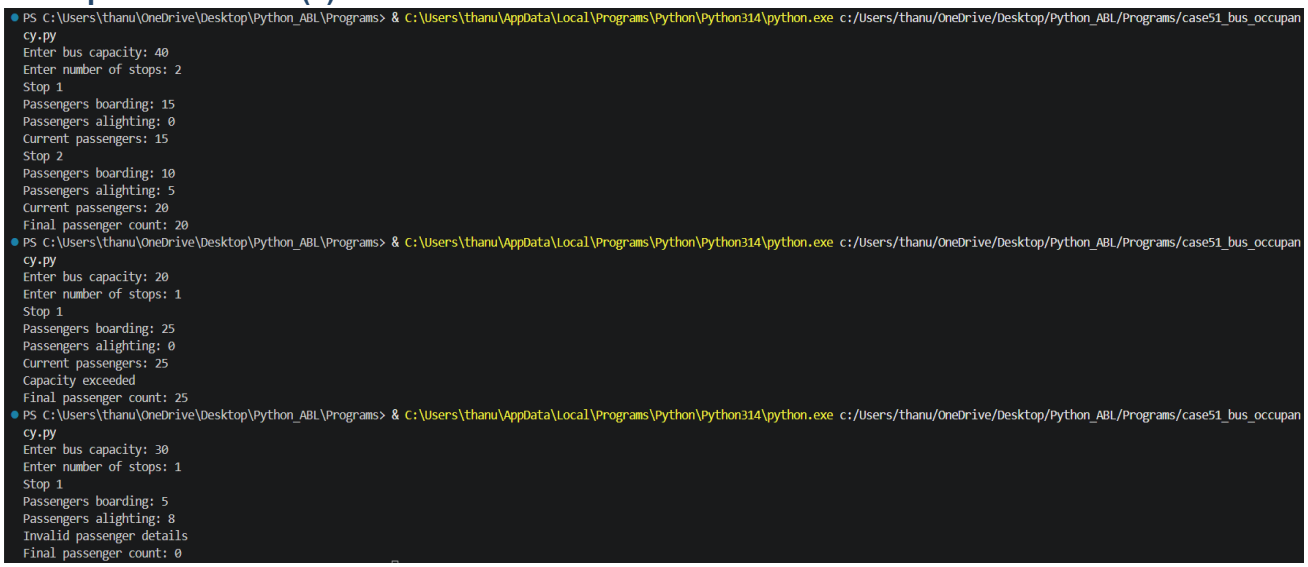
8. Test Cases

• Test 1: Capacity 40, stops: board 15/alight 0; board 10/alight 5 -> final 20

• Test 2: Capacity 20; board 25/alight 0 -> capacity exceeded; final 25

• Test 3: Capacity 30; board 5/alight 8 -> invalid passenger details; final 0

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program tracks passenger occupancy across stops, checks passenger details, and warns when the bus capacity is exceeded.

Case 57: Smart Canteen Order and Stock Management

Python file: **case57_canteen.py**

1. Problem Statement

Process canteen orders, check menu availability and stock, calculate accepted order costs, and report total sales and remaining stock.

2. Objectives

• Maintain menu prices and available stock.

• Validate items and quantities.

• Calculate sales and update stock only for accepted orders.

3. Assumptions

• The menu contains idli, dosa, and juice.

• Prices and starting stock are predefined in the program.

• Quantity must be positive.

• Orders exceeding available stock are rejected.

4. Algorithm

1. Start.

2. Define menu prices and stock.

3. Read number of orders.

4. For each order, read item and quantity.

5. Check item, quantity, and stock.

6. If valid, calculate cost, add to sales, and reduce stock.

7. Display total sales and final stock.

8. Stop.

5. Flowchart

START → Set menu and stock → Read order count → Read item and quantity → Valid item? → Valid quantity/stock? → Accept and update totals/stock or reject → Print summary → STOP

6. Pseudocode

```
1. Create dictionaries for menu prices and stock; set total sales to zero.
2. Read number of orders.
3. For each order, read item and quantity.
4. Reject unavailable items, non-positive quantities, or insufficient stock.
5. Otherwise calculate cost, update total sales, reduce stock, and print order details.
6. Print total sales and final stock.
```

7. Python Source Code

```
menu = {"idli": 20, "dosa": 50, "juice": 40} stock
= {"idli": 20, "dosa": 10, "juice": 8} total_sales
= 0
n = int(input("Enter number of orders: ")) for
i in range(n):
    print("Order", i + 1)     item = input("Enter item
(idli/dosa/juice): ").lower()     quantity =
int(input("Enter quantity: "))     if item not in menu:
        print("Item not available")
elif quantity <= 0:
        print("Invalid quantity")
elif quantity > stock[item]:
print("Insufficient stock")     else:
        cost = menu[item] * quantity
total_sales += cost         stock[item] -=
quantity         print("Order accepted. Cost:
Rs.", cost)         print("Remaining stock:",
stock[item]) print("Total sales: Rs.",
total_sales) print("Final stock:", stock)
```

8. Test Cases

• Test 1: idli 3, dosa 2, juice 2 -> sales Rs. 240; stock 17, 8, 6

• Test 2: juice 10, idli 2 -> juice rejected; idli accepted Rs. 40; sales Rs. 40

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case57_canteen.py
Enter number of orders: 3
Order 1
Enter item (idli/dosa/juice): idli
Enter quantity: 3
Order accepted. Cost: Rs. 60
Remaining stock: 17
Order 2
Enter item (idli/dosa/juice): dosa
Enter quantity: 2
Order accepted. Cost: Rs. 100
Remaining stock: 8
Order 3
Enter item (idli/dosa/juice): juice
Enter quantity: 2
Order accepted. Cost: Rs. 80
Remaining stock: 6
Total sales: Rs. 240
Final stock: {'idli': 17, 'dosa': 8, 'juice': 6}
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case57_canteen.py
Enter number of orders: 2
Order 1
Enter item (idli/dosa/juice): juice
Enter quantity: 10
Insufficient stock
Order 2
Enter item (idli/dosa/juice): idli
Enter quantity: 2
Order accepted. Cost: Rs. 40
Remaining stock: 18
Total sales: Rs. 40
Final stock: {'idli': 18, 'dosa': 10, 'juice': 8}
```

10. Conclusion

The program processes canteen orders, validates stock, updates inventory, and calculates total sales.

Case 66: Student Academic Risk Classification

Python file: **case66_academic_risk.py**

1. Problem Statement

Classify student academic risk using marks, attendance, and lab score, and report the risk factors detected.

2. Objectives

• Read student academic details.

• Apply predefined risk conditions.

• Report risk factors for each student.

3. Assumptions

• Marks, attendance, and lab score are entered as percentages or scores on a 0–100 scale.

• Risk is identified using the thresholds implemented in the source code.

• The classification is a simple educational example, not an official institutional decision.

4. Algorithm

1. Start.

2. Read number of students.

3. For each student, read name, marks, attendance, and lab score.

4. Check each value against the risk thresholds.

5. Display the risk factors or report no identified risk.

6. Stop.

5. Flowchart

START → Read student count → Read each student's details → Check marks, attendance, lab score → Display risk factors/classification → STOP

6. Pseudocode

```
1. Read number of students.
2. For each student, read name, marks, attendance, and lab score.
3. Check each input against the program's risk thresholds.
4. Collect and display all risk factors found.
5. If no factor is found, report no academic risk.
```

7. Python Source Code

```
n = int(input("Enter number of students: "))
risk_count = 0 for i in range(n):
    print("Student", i + 1)     name = input("Enter student
name: ")     marks = float(input("Enter marks percentage: "))
attendance = float(input("Enter attendance percentage: "))
lab = float(input("Enter lab score percentage: "))     risks =
[]     if marks < 50:
        risks.append("Low marks")
if attendance < 75:
        risks.append("Low attendance")
if lab < 50:
        risks.append("Low lab score")
if risks:
        print(name, "- At academic risk:", ", ".join(risks))
risk_count += 1     else:
        print(name, "- No academic risk") print("Students
at academic risk:", risk_count)
```

8. Test Cases

• Test 1: Anu: marks 80, attendance 90, lab 85 -> no academic risk

• Test 2: Ravi: 45, 80, 70 -> low marks

• Test 3: Kiran: 70, 65, 40 -> low attendance and low lab score

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program checks student academic details against defined thresholds and reports the risk factors found.

Case 87: Examination Question Difficulty and Performance Analysis

Python file: **case87_exam_analysis.py**

1. Problem Statement

Compare students' answer strings with an answer key, calculate student scores and question-wise success rates, and identify the hardest questions.

2. Objectives

• Validate answer string lengths.

• Calculate student scores and percentages.

• Calculate question-wise success rates and identify the lowest-performing questions.

3. Assumptions

• The answer key and student responses use one character per question.

• Answer strings must have the same length as the answer key.

• A response is correct when its character matches the key at the same position.

4. Algorithm

1. Start.

2. Read question count and answer key.

3. Read each student's answer string.

4. Validate its length.

5. For valid answers, calculate score and update per-question correct counts.

6. Calculate question-wise success rates.

7. Identify question(s) with the lowest success rate.

8. Display results and stop.

5. Flowchart

START → Read question count and key → Read student answers → Correct length? → Score valid submissions → Calculate question success rates → Identify hardest question(s) → STOP

6. Pseudocode

```
1. Read number of questions and answer key.
2. Read number of students and each answer string.
3. Reject responses with incorrect length.
4. For valid responses, compare each answer with the key and calculate the score.
5. Calculate each question's success percentage.
6. Display student results, question-wise rates, and the hardest question(s).
```

7. Python Source Code

```
n = int(input("Enter number of questions: ")) if
n <= 0:
    print("Invalid number of questions") else:
    key = input("Enter answer key: ").upper()
if len(key) != n:
        print("Answer key length must match number of questions")
else:
        students = int(input("Enter number of students: "))
if students <= 0:
            print("Invalid number of students")
else:
            correct_counts = [0] * n
valid_submissions = 0             for i
in range(students):
print("Student", i + 1)
                answers = input("Enter answers: ").upper()
if len(answers) != n:
                    print("Invalid answer length; submission skipped")
```

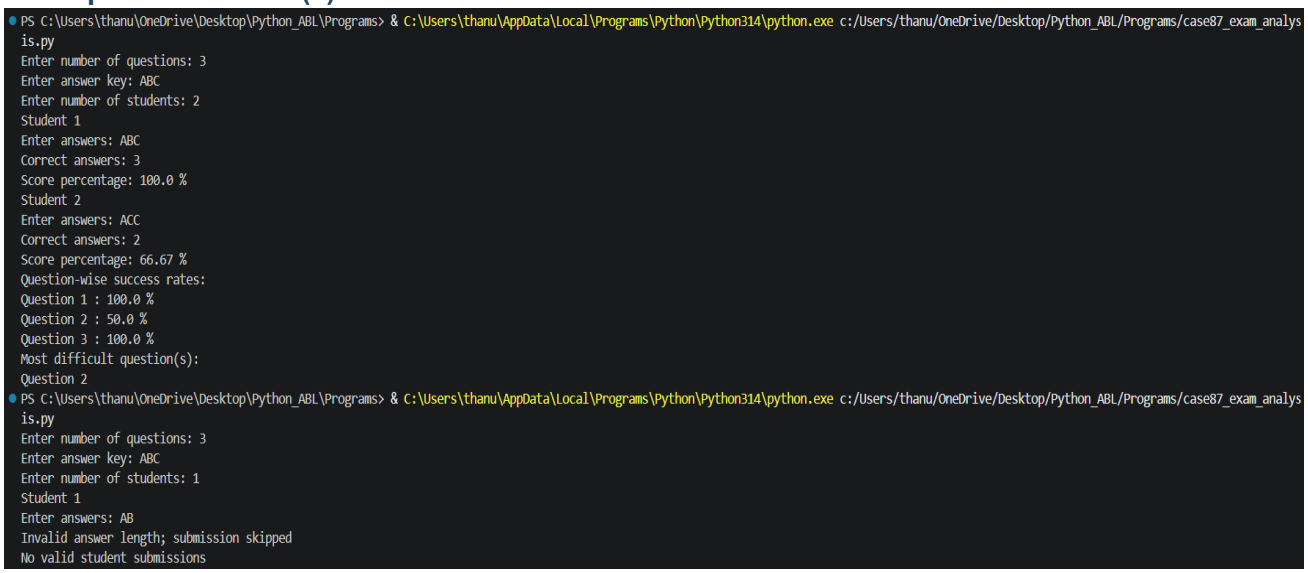
```
                    continue                valid_submissions += 1
score = 0                 for j in range(n):                    if
answers[j] == key[j]:                        score += 1
correct_counts[j] += 1                print("Correct answers:", score)
print("Score percentage:", round(score / n * 100, 2), "%")             if
valid_submissions == 0:
                print("No valid student submissions")
else:
                print("Question-wise success rates:")
for j in range(n):
                    rate = correct_counts[j] / valid_submissions * 100
print("Question", j + 1, ":", round(rate, 2), "%")
lowest = min(correct_counts)                print("Most difficult
question(s):")                for j in range(n):                    if
correct_counts[j] == lowest:                        print("Question", j
+ 1)
```

8. Test Cases

• Test 1: n=3, key ABC, answers ABC and ACC -> scores 3 (100%) and 2 (66.67%); Q1=100%, Q2=50%, Q3=50%; hardest Q2 and Q3

• Test 2: n=3, key ABC, one answer AB -> invalid answer length; no valid student submissions

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



10. Conclusion

The program compares answer strings with the key, calculates scores and question-wise success rates, and identifies the hardest questions.

Case 91: Advanced String-Based Registration Record Analyzer

Python file: **case91_registration.py**

1. Problem Statement

Read registration records separated by vertical bars, validate registration IDs and email addresses, detect duplicate IDs, and count valid and invalid records.

2. Objectives

• Split each record into fields.

• Validate ID and email formats.

• Detect duplicate registration IDs and summarise records.

3. Assumptions

• Each record is entered on one line as ID|Name|Email.

• Registration IDs and email addresses must follow the checks implemented in the program.

• Duplicate IDs are treated as invalid records.

4. Algorithm

1. Start.

2. Read number of records.

3. For each record, split the line using the | separator.

4. Validate the number of fields, ID, email, and duplicate status.

5. Count valid and invalid records.

6. Display summary and stop.

5. Flowchart

START → Read record count → Read ID|Name|Email line → Split fields → Validate fields and duplicate ID → Count valid/invalid → Print summary → STOP

6. Pseudocode

```
1. Read number of records.
2. Initialise the set of IDs and valid, invalid, and duplicate counters.
3. For each line, split using '|'.
4. Check field count, ID format, email format, and whether ID is repeated.
5. Count and report invalid or duplicate records as implemented.
6. Print valid, invalid, and duplicate totals.
```

7. Python Source Code

```
n = int(input("Enter number of registration records: ")) if
n <= 0:
    print("Invalid number of records") else:
    seen = set()     valid = invalid
= duplicates = 0     for i in
range(n):
        print("Record", i + 1)         record =
input("Enter ID|Name|Email: ")         parts =
record.split("|")         if len(parts) != 3:
print("Invalid record format")
invalid += 1             continue
        reg_id = parts[0].strip().upper()         name = parts[1].strip()         email =
parts[2].strip()         errors = []         if len(reg_id) != 6 or not
reg_id.startswith("REG") or not reg_id[3:].isdigit():
            errors.append("Invalid registration ID")
if name == "":
            errors.append("Missing name")         if "@" not in
email or "." not in email.split("@")[-1]:
            errors.append("Invalid email")
if reg_id in seen:
            print("Duplicate registration ID:", reg_id)
            duplicates += 1
errors.append("Duplicate ID")         else:
```

```
            seen.add(reg_id)
if errors:              for error
in errors:
                  print(error)
invalid += 1          else:
            print("Valid record:", reg_id, name, email)
valid += 1      print("Valid records:", valid)
print("Invalid records:", invalid)      print("Duplicate
IDs detected:", duplicates)
```

8. Test Cases

• Test 1: REG001|Anu|anu@gmail.com; REG002|Ravi|ravi@gmail.com -> 2 valid, 0 invalid, 0 duplicates

• Test 2: R12|Anu|anugmail.com -> invalid ID and email; 0 valid, 1 invalid, 0 duplicates

• Test 3: REG001|Anu|anu@gmail.com; REG001|Ravi|ravi@gmail.com -> 1 valid, 1 invalid, 1 duplicate

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case91_registration.py
Enter number of registration records: 2
Record 1
Enter ID|Name|Email: REG001|Anu|anu@gmail.com
Valid record: REG001 Anu anu@gmail.com
Record 2
Enter ID|Name|Email: REG002|Ravi|ravi@gmail.com
Valid record: REG002 Ravi ravi@gmail.com
Valid records: 2
Invalid records: 0
Duplicate IDs detected: 0
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case91_registration.py
Enter number of registration records: 1
Record 1
Enter ID|Name|Email: R12|Anu|anugmail.com
Invalid registration ID
Invalid email
Valid records: 0
Invalid records: 1
Duplicate IDs detected: 0
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case91_registration.py
Enter number of registration records: 2
Record 1
Enter ID|Name|Email: REG001|Anu|anu@gmail.com
Valid record: REG001 Anu anu@gmail.com
Record 2
Enter ID|Name|Email: REG001|Rabi|ravi@gmail.com
Duplicate registration ID: REG001
Duplicate ID
Valid records: 1
Invalid records: 1
Duplicate IDs detected: 1
```

10. Conclusion

The program analyses structured registration strings, validates fields, detects duplicate IDs, and summarises valid and invalid records.

Case 93: Smart Public Transport Fare Reconciliation

Python file: **case93_fare_reconciliation.py**

1. Problem Statement

Calculate expected fares for route and passenger-category records, compare them with collected fares, and report discrepancies and totals.

2. Objectives

• Use predefined fares for routes and passenger categories.

• Validate route, category, passenger count, and collected fare.

• Compare expected and collected revenue.

3. Assumptions

• Route A and B fares are predefined in the program.

• Passenger count must be positive.

• Collected fare cannot be negative.

• A discrepancy is the collected fare minus the expected fare.

4. Algorithm

1. Start.

2. Define fare table.

3. Read number of route records.

4. For each record, read route, category, passenger count, and collected fare.

5. Validate input.

6. Calculate expected fare and difference.

7. Report reconciliation or discrepancy.

8. Display totals and stop.

5. Flowchart

START → Define fares → Read record count → Read route/category/passengers/collected fare → Validate → Calculate expected fare and difference → Print record result → Print totals → STOP

6. Pseudocode

```
1. Define the fare table for routes A and B.
2. Read number of records and initialise totals/counter.
3. For each record, read route, category, passengers, and collected fare.
4. Reject invalid route, category, passenger count, or fare.
5. Calculate expected fare and difference = collected - expected.
6. Update totals; report reconciled fare or discrepancy.
7. Print expected total, collected total, and discrepancy count.
```

7. Python Source Code

```
fares = {     "A": {"adult": 20, "student": 10},
"B": {"adult": 30, "student": 15} } n =
int(input("Enter number of route records: "))
expected_total = collected_total = 0 discrepancies
= 0 if n <= 0:
    print("Invalid number of records") else:
    for i in range(n):
        print("Record", i + 1)
        route = input("Enter route A or B: ").upper()
category = input("Enter adult or student: ").lower()
passengers = int(input("Enter passenger count: "))
collected = float(input("Enter collected fare: "))         if
route not in fares:             print("Invalid route")
        elif category not in fares[route]:
            print("Invalid passenger category")
elif passengers <= 0 or collected < 0:
```

```
            print("Invalid passenger count or fare")
else:
            expected = fares[route][category] * passengers
difference = collected - expected             expected_total
+= expected             collected_total += collected
print("Expected fare: Rs.", expected)
print("Collected fare: Rs.", round(collected, 2))
if difference == 0:
                print("Fare reconciled")
else:
                print("Discrepancy: Rs.", round(difference, 2))
discrepancies += 1     print("Total expected fare: Rs.",
expected_total)     print("Total collected fare: Rs.",
round(collected_total, 2))     print("Records with discrepancies:",
discrepancies)
```

8. Test Cases

- Test 1: Route A adult 2 collected 40; Route B student 2 collected 30 -> expected total Rs. 70, collected Rs. 70.0, discrepancies 0
- Test 2: Route A adult 3 collected 50 -> expected Rs. 60, discrepancy Rs. -10.0, discrepancies 1
- Test 3: Route B student 2 9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case93_fare_reconc
iliation.py
Enter number of route records: 2
Record 1
Enter route A or B: A
Enter adult or student: adult
Enter passenger count: 2
Enter collected fare: 40
Expected fare: Rs. 40
Collected fare: Rs. 40.0
Fare reconciled
Record 2
Enter route A or B: B
Enter adult or student: student
Enter passenger count: 2
Enter collected fare: 30
Expected fare: Rs. 30
Collected fare: Rs. 30.0
Fare reconciled
Total expected fare: Rs. 70
Total collected fare: Rs. 70.0
Records with discrepancies: 0
PS C:\Users\thanu\OneDrive\Desktop\Python_ABL\Programs> & C:\Users\thanu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/thanu/OneDrive/Desktop/Python_ABL/Programs/case93_fare_reconc
iliation.py
Enter number of route records: 1
Record 1
Enter route A or B: C
Enter adult or student: adult
Enter passenger count: 2
Enter collected fare: 40
Invalid route
Total expected fare: Rs. 0
Total collected fare: Rs. 0
Records with discrepancies: 0
```

10. Conclusion

The program calculates expected transport fares, compares them with collected amounts, and reports revenue discrepancies.